

SuperBizAgent 面试 Q&A 文档

适用对象：大二学生，面试 Java + AI 岗位

问题梯度：基础概念 → 项目实现 → 架构设计 → 优化扩展



第一部分：项目基础问题（必答）

Q1: 请用1分钟简单介绍一下这个项目

答案：

这是一个基于 **Spring Boot 3.2 + Spring AI Alibaba** 的企业级智能业务代理系统，主要包含两大核心功能：

- RAG 智能问答**：用户可以上传文档（txt、md），系统会自动切分、向量化并存储到 Milvus 向量数据库。当用户提问时，系统会检索最相关的 top-3 文档片段，结合阿里云 DashScope 大模型生成准确答案，避免模型幻觉。
- AIOps 智能运维**：基于多 Agent 协作（ReactAgent + SupervisorAgent）的自动化运维系统，采用 Planner-Executor-Replanner 架构，能够自动查询告警、分析日志、检索内部文档，最终生成 Markdown 格式的运维分析报告。

技术栈包括 Java 17、Spring Boot、Spring AI、Milvus 向量数据库、阿里云 DashScope、SSE 流式输出等。

Q2: 为什么选择做这个项目？你从中学到了什么？

答案：

我选择这个项目的原因有三点：

- 技术前沿性**：RAG 和 Agent 是当前 AI 落地的热点方向，想要深入理解这两项技术
- 工程实践性**：不是简单调 API，而是涉及数据处理、并发控制、流式输出等工程问题
- 实际应用价值**：解决真实场景问题（知识检索、智能运维），有落地意义

主要收获：

- 掌握了 **RAG 技术栈**：文档切分、向量化、相似度检索、Prompt Engineering
- 理解了 **AI Agent 设计模式**：工具调用、多 Agent 编排、状态管理
- 提升了 **Spring Boot 工程能力**：SSE 流式接口、会话管理、并发安全
- 学会了如何将 AI 能力与实际业务场景结合

Q3: 项目的核心技术栈有哪些？为什么选择它们？

答案：

技术	版本	选择理由
Java 17	17	LTS 版本，支持 Record、Pattern Matching 等新特性
Spring Boot	3.2.0	成熟的企业级框架，快速搭建 RESTful API
Spring AI Alibaba	1.1.0.0-RC2	阿里云官方 AI 框架，集成 DashScope 更简单

Spring AI Alibaba 技术 Milvus	1.1.0-RC2 版本 2.6.10	阿里云官方 AI 框架，集成 DashScope 更简单 选择理由 开源向量数据库，支持高性能相似度搜索
DashScope SDK	2.17.0	阿里云 AI 服务，提供大模型和 Embedding 能力
Lombok	1.18.30	简化 Java 代码，减少样板代码

为什么选 Milvus 而不是其他向量数据库？

- 开源免费，本地部署方便（Docker Compose 一键启动）
- 性能好，支持百万级向量检索
- 社区活跃，文档完善
- 与主流 AI 框架集成方便

 第二部分：技术实现问题（核心）

Q4: 什么是 RAG？为什么需要 RAG？

答案：
RAG（Retrieval-Augmented Generation，检索增强生成）是一种结合检索和生成的 AI 技术。

核心流程：

1. 离线阶段：文档 → 切分 → 向量化 → 存入向量数据库
2. 在线阶段：用户问题 → 向量化 → 检索相似文档 → 拼接到 Prompt → 大模型生成答案

为什么需要 RAG？

问题	直接用大模型	使用 RAG
知识过时	X 训练数据有截止日期	✓ 实时更新企业知识库
专业领域知识不足	X 通用模型缺少垂直领域知识	✓ 注入企业内部文档
幻觉问题	X 可能编造不存在的信息	✓ 基于真实文档生成答案
隐私安全	X 数据上传到训练集风险	✓ 私有部署，数据可控

在本项目中的体现：

```
// RagService.java 核心流程
public void queryStream(String question, List<Map<String, String>> history,
streamCallback callback) {
    // 1. 向量检索相关文档
    List<SearchResult> searchResults =
vectorSearchService.searchSimilarDocuments(question, topK);

    // 2. 构建上下文
    String context = buildContext(searchResults);

    // 3. 构建 Prompt（注入检索到的文档）
    String prompt = buildPrompt(question, context);

    // 4. 调用大模型生成答案
```

```
generateAnswerStream(prompt, history, callback);  
}
```

Q5: 文档是如何被切分和向量化的?

答案:

1 文档切分 (DocumentChunkService)

为什么要切分?

- 大模型有 **token 限制** (例如 qwen3-max 支持 8k tokens)
- 大段文本不利于 **精准检索**
- 切分后可以 **按章节/段落** 定位信息

切分策略:

```
# application.yml  
document:  
  chunk:  
    max-size: 800      # 每个分片最大 800 字符  
    overlap: 100       # 相邻分片重叠 100 字符
```

为什么要 overlap?

- 防止关键信息被切断
- 例如: "根据公司规定, 员工请假需提前|3天申请" → 如果在 | 处切断, 前一段缺少"3天", 后一段缺少"请假"上下文

代码实现:

```
// DocumentChunkService.java  
public List<String> chunkDocument(String content) {  
    List<String> chunks = new ArrayList<>();  
    int start = 0;  
  
    while (start < content.length()) {  
        int end = Math.min(start + maxSize, content.length());  
        chunks.add(content.substring(start, end));  
  
        // 下一段从 (start + maxSize - overlap) 开始  
        start += (maxSize - overlap);  
    }  
  
    return chunks;  
}
```

2 向量化 (VectorEmbeddingService)

什么是向量化 (Embedding) ?

将文本转换为高维向量 (数字数组), 语义相近的文本向量距离更近。

使用的模型:

```
dashscope:
  embedding:
    model: text-embedding-v4 # 阿里云 1536 维向量
```

代码实现:

```
// VectorEmbeddingService.java
public List<Float> generateEmbedding(String text) {
    TextEmbeddingParam param = TextEmbeddingParam.builder()
        .model("text-embedding-v4")
        .texts(Arrays.asList(text))
        .build();

    TextEmbeddingResult result = embeddingService.call(param);
    return result.getOutput().getEmbeddings().get(0).getEmbedding();
}
```

3 存储到 Milvus

```
// VectorIndexService.java
public void indexDocument(String filename, String content) {
    // 1. 切分文档
    List<String> chunks = chunkService.chunkDocument(content);

    // 2. 批量向量化
    for (String chunk : chunks) {
        List<Float> vector = embeddingService.generateEmbedding(chunk);

        // 3. 插入 Milvus
        InsertParam insertParam = InsertParam.builder()
            .collectionName("documents")
            .fields(List.of(
                new InsertParam.Field("id", UUID.randomUUID().toString()),
                new InsertParam.Field("vector", vector),
                new InsertParam.Field("content", chunk),
                new InsertParam.Field("filename", filename)
            ))
            .build();
        milvusClient.insert(insertParam);
    }
}
```

Q6: 向量检索是如何工作的? top-k 怎么理解?

答案:

向量检索原理

核心思想: 用 **余弦相似度** 或 **欧氏距离** 计算查询向量与库中所有向量的相似度, 返回最相似的 k 个结果。

余弦相似度公式:

$$\text{similarity}(A, B) = (A \cdot B) / (||A|| * ||B||)$$

- 值域: $[-1, 1]$, 越接近 1 越相似
- 只关注方向, 不关注长度 (适合文本语义)

在 Milvus 中的实现:

```
// VectorSearchService.java
public List<SearchResult> searchSimilarDocuments(String query, int topK) {
    // 1. 查询文本向量化
    List<Float> queryVector = embeddingService.generateQueryVector(query);

    // 2. Milvus 向量搜索
    SearchParam searchParam = SearchParam.builder()
        .collectionName("documents")
        .vectorFieldName("vector")
        .vectors(Collections.singletonList(queryVector))
        .topK(topK) // 返回最相似的 3 个结果
        .metricType(MetricType.COSINE) // 余弦相似度
        .build();

    R<SearchResults> response = milvusClient.search(searchParam);

    // 3. 解析结果
    SearchResultsWrapper wrapper = new SearchResultsWrapper(response.getData());
    return parseResults(wrapper);
}
```

top-k 的理解

top-k=3 表示返回最相似的 3 个文档片段。

为什么设置为 3?

- 太少 (k=1) : 信息可能不全面
- 太多 (k>5) :
 - 引入噪声 (不相关的文档)
 - 超过大模型 context 长度
 - 生成速度变慢

配置:

```
rag:
  top-k: 3 # 可以根据实际效果调整
```

Q7: 什么是 AI Agent? 为什么需要 Agent?

答案:

AI Agent 定义

Agent（智能体）是指能够：

- 1. **感知环境**：接收输入、读取状态
- 2. **自主决策**：根据目标制定计划
- 3. **执行动作**：调用工具、修改环境
- 4. **迭代优化**：根据反馈调整策略

与传统 AI 的区别：

传统 AI	AI Agent
一问一答	多轮交互
被动响应	主动规划
单一能力	工具调用
静态回答	动态推理

为什么需要 Agent？

场景举例：智能运维排障

✖ 传统方式（人工）：

- 1. 查看 Prometheus 告警 → 手动登录监控平台
- 2. 分析告警原因 → 手动查询日志
- 3. 查阅运维文档 → 手动搜索 wiki
- 4. 编写分析报告 → 手动整理文档

✔ Agent 方式（自动）：

用户：分析当前的告警情况

Agent 执行流程：

- 1. 调用 queryPrometheusAlerts() 获取告警列表
- 2. 对每个告警，调用 queryLogs() 查询相关日志
- 3. 调用 queryInternalDocs() 搜索处理方案
- 4. 自动生成 Markdown 分析报告

在本项目中的体现：

```
// ChatService.java - ReactAgent 配置
ReactAgent agent = ReactAgent.builder()
    .name("intelligent_assistant")
    .model(chatModel)
    .systemPrompt(systemPrompt)
    .methodTools(new Object[]{
        dateTimeTools,           // 时间工具
        internalDocsTools,       // 文档检索工具
        queryMetricsTools,       // 告警查询工具
        queryLogsTools           // 日志查询工具
    })
```

```
.tools(toolCallbacks) // MCP 外部工具
.build();
```

```
// Agent 会根据用户问题自动选择合适的工具调用
String answer = agent.call("查询最近的告警并分析原因");
```

Q8: 什么是 ReactAgent? 它是如何工作的?

答案:

ReactAgent 原理

React = Reasoning + Acting (推理 + 行动)

核心流程:

```
用户输入
↓
【Thought】思考：我需要做什么？
↓
【Action】行动：调用工具
↓
【Observation】观察：工具返回结果
↓
【Thought】思考：结果是否满足目标？
├ 是 → 【Answer】返回最终答案
└ 否 → 回到 Action（继续调用工具）
```

代码实现:

```
// ChatService.java
public ReactAgent createReactAgent(DashScopeChatModel chatModel, String
systemPrompt) {
    return ReactAgent.builder()
        .name("intelligent_assistant")
        .model(chatModel)
        .systemPrompt(systemPrompt) // 指导 Agent 行为
        .methodTools(buildMethodToolsArray()) // 本地 Java 方法工具
        .tools(getToolCallbacks()) // MCP 外部工具
        .build();
}

// 执行对话
String answer = agent.call("当前时间是多少? ");

// Agent 内部执行过程:
// 1. 【Thought】用户问时间，我需要调用 getCurrentDateTime 工具
// 2. 【Action】调用 dateTimeTools.getCurrentDateTime()
// 3. 【Observation】返回 "2026-04-08T11:15:06.323Z"
// 4. 【Thought】已获取时间，可以回答了
// 5. 【Answer】"当前时间是 2026 年 4 月 8 日 11:15"
```

工具调用机制

工具定义示例：

```
// DateTimeTools.java
@Component
public class DateTimeTools {

    @Tool(description = "Get the current date and time in the user's timezone")
    public String getCurrentDateTime() {
        return LocalDateTime.now()
            .atZone(LocaleContextHolder.getTimeZone().toZoneId())
            .toString();
    }
}
```

Agent 如何选择工具？

1. **系统提示词** 告诉 Agent 什么时候用什么工具

```
String systemPrompt = """
你是一个专业的智能助手，可以：
- 当用户询问时间相关问题时，使用 getCurrentDateTime 工具
- 当用户需要查询内部文档时，使用 queryInternalDocs 工具
- 当用户需要查询告警时，使用 queryPrometheusAlerts 工具
""";
```

2. **工具描述** (@Tool description) 帮助模型理解工具用途
3. **模型推理** 决定是否调用、调用哪个工具、传入什么参数

Q9: 多 Agent 协作是怎么实现的？Planner 和 Executor 的职责是什么？

答案：

多 Agent 架构

Planner-Executor-Replanner 模式



为什么要拆成多个 Agent？

单 Agent 问题	多 Agent 优势
规划、执行、总结混在一起	职责清晰，每个 Agent 专注一件事
容易出现计划不可执行	Planner 制定计划，Executor 验证可行性
难以处理复杂长流程	可迭代执行，动态调整计划
错误难以定位	每个 Agent 的输出可审计

具体实现

1 Planner Agent (规划者)

```
// AiOpsService.java
ReactAgent plannerAgent = ReactAgent.builder()
    .name("planner_agent")
    .description("负责拆解告警、规划与再规划步骤")
    .model(chatModel)
    .systemPrompt(buildPlannerPrompt())
    .methodTools(buildMethodToolsArray())
    .tools(toolCallbacks)
    .outputKey("planner_plan") // 输出写入状态机的 planner_plan 字段
    .build();
```

职责：

- 读取任务 {input} 和执行反馈 {executor_feedback}
- 制定下一步计划 (JSON 格式)

```
{
  "decision": "EXECUTE",
  "step": "查询 Prometheus 告警",
  "tool": "queryPrometheusAlerts",
  "reason": "需要先了解当前活跃告警"
}
```

- 当所有步骤完成后，输出 decision=FINISH 并生成最终报告

2 Executor Agent (执行者)

```
ReactAgent executorAgent = ReactAgent.builder()
    .name("executor_agent")
    .description("负责执行 Planner 的首个步骤并及时反馈")
    .model(chatModel)
    .systemPrompt(buildExecutorPrompt())
    .methodTools(buildMethodToolsArray())
    .tools(toolCallbacks)
    .outputKey("executor_feedback") // 输出写入 executor_feedback 字段
    .build();
```

职责：

- 读取 Planner 的计划 {planner_plan}

- 只执行第一步（避免发散）
- 调用相应工具，收集证据
- 返回执行结果（JSON 格式）

```
{
  "status": "SUCCESS",
  "summary": "查询到 3 条活跃告警",
  "evidence": "[告警详情...]",
  "nextHint": "建议查询日志确认根因"
}
```

3 Supervisor Agent（调度者）

```
SupervisorAgent supervisorAgent = SupervisorAgent.builder()
    .name("ai_ops_supervisor")
    .description("负责调度 Planner 与 Executor 的多 Agent 控制器")
    .model(chatModel)
    .systemPrompt(buildSupervisorSystemPrompt())
    .subAgents(List.of(plannerAgent, executorAgent))
    .build();

// 执行任务
Optional<OverAllState> result = supervisorAgent.invoke(taskPrompt);
```

职责：

- 决定何时调用 Planner、何时调用 Executor
- 管理状态流转
- 检测是否达到终止条件（decision=FINISH）

执行流程示例

用户：分析当前的告警情况

```
Supervisor: 调用 Planner
↓
Planner: {"decision": "EXECUTE", "step": "查询告警"}
↓
Supervisor: 调用 Executor
↓
Executor: {"status": "SUCCESS", "summary": "查到 3 条告警"}
↓
Supervisor: 再次调用 Planner
↓
Planner: {"decision": "EXECUTE", "step": "查询日志"}
↓
Supervisor: 调用 Executor
↓
Executor: {"status": "SUCCESS", "summary": "日志显示 OOM 错误"}
↓
Supervisor: 再次调用 Planner
↓
Planner: {"decision": "FINISH", "report": "# 告警分析报告\n..."}
↓
```

Q10: 项目中的并发问题是如何处理的?

答案:

问题背景

用户可能同时发起多个对话请求，需要保证：

1. **会话隔离**：不同用户的对话不能互相干扰
2. **线程安全**：同一会话的并发请求不能产生数据竞争
3. **历史消息一致性**：读写历史消息时不能出现错乱

解决方案

1 使用 ConcurrentHashMap 存储会话

```
// ChatController.java
private final Map<String, SessionInfo> sessions = new ConcurrentHashMap<>();
```

为什么用 ConcurrentHashMap?

- 线程安全的 Map
- 支持高并发读写
- 不会在迭代时抛出 `ConcurrentModificationException`

2 每个会话使用 ReentrantLock

```
@Getter
@Setter
public static class SessionInfo {
    private List<Map<String, String>> history;
    private final ReentrantLock lock; // 会话级别的锁

    public SessionInfo() {
        this.history = new ArrayList<>();
        this.lock = new ReentrantLock();
    }
}
```

为什么需要 ReentrantLock?

- ConcurrentHashMap 只保证 Map 本身线程安全
- **无法保证复合操作的原子性**（例如：读取历史 → 添加消息 → 保存历史）

3 加锁保护临界区

```
// ChatController.java - chat 接口
SessionInfo session = getOrCreateSession(request.getId());

try {
    session.getLock().lock(); // 获取锁
```

```
// 临界区：读写历史消息
List<Map<String, String>> history = session.getHistory();

// ... 执行对话 ...

// 更新会话历史（滑动窗口保留最近 6 对消息）
history.add(Map.of("role", "user", "content", request.getQuestion()));
history.add(Map.of("role", "assistant", "content", fullAnswer));
maintainwindow(history, MAX_WINDOW_SIZE);

} finally {
    session.getLock().unlock(); // 释放锁
}
```

为什么用 try-finally?

- 确保即使发生异常，也能释放锁
- 避免死锁

4 滑动窗口维护历史

```
private void maintainwindow(List<Map<String, String>> history, int
maxWindowSize) {
    // 保留最近的 maxWindowSize 对（用户+助手）消息
    while (history.size() > maxWindowSize * 2) {
        history.remove(0); // 移除最早的用户消息
        history.remove(0); // 移除最早的助手消息
    }
}
```

为什么要限制历史长度?

- 避免 token 超限（大模型有输入长度限制）
- 减少内存占用
- 提高推理速度

并发场景测试

场景 1：不同用户同时对话

用户 A (session-a): 查询天气
用户 B (session-b): 查询告警

✅ **结果：**两个会话独立，互不干扰

场景 2：同一用户快速连发

用户 (session-123):
- 请求 1: 查询天气
- 请求 2: 查询告警（在请求 1 未完成时发出）

✅ **结果：**

- 请求 2 会等待请求 1 释放锁
- 请求 2 能看到请求 1 产生的历史消息

Q11: SSE (Server-Sent Events) 流式输出是如何实现的？

答案：

什么是 SSE？

SSE (Server-Sent Events) 是一种服务器向客户端推送数据的技术。

与 WebSocket 的区别：

特性	SSE	WebSocket
通信方向	单向（服务器→客户端）	双向
协议	HTTP	WebSocket
实现复杂度	简单	复杂
浏览器支持	所有现代浏览器	需检查兼容性
适用场景	日志推送、进度更新	实时聊天、游戏

为什么要用流式输出？

- 大模型生成速度慢（可能需要 10-30 秒）
- 流式输出能让用户**实时看到进展**，减少等待焦虑
- 适合长文本生成（逐字输出）

实现方式 1：Spring SseEmitter

接口定义：

```
// ChatController.java
@PostMapping(value = "/chat_stream", produces =
MediaType.TEXT_EVENT_STREAM_VALUE)
public SseEmitter chatStream(@RequestBody ChatRequest request) {
    SseEmitter emitter = new SseEmitter(180000L); // 超时 3 分钟

    executor.submit(() -> {
        try {
            // 构建 Agent
            ReactAgent agent = chatService.createReactAgent(chatModel,
systemPrompt);

            // 获取流式输出
            Flux<StreamingOutput> stream = agent.stream(request.getQuestion());

            // 订阅流式数据
            stream.subscribe(
                output -> {
                    // 每收到一个 token，就推送给客户端
                    if (output.type() == OutputType.AGENT_MODEL_STREAMING) {
                        emitter.send(SseEmitter.event()
                            .name("content")
                            .data(output.content()));
                    }
                }
            );
        } catch (Exception e) {
            emitter.completeWithError(e);
        }
    });
}
```

```

    }
    },
    error -> {
        emitter.send(SseEmitter.event()
            .name("error")
            .data("Error: " + error.getMessage()));
        emitter.complete();
    },
    () -> {
        emitter.send(SseEmitter.event().name("done").data("[DONE]"));
        emitter.complete();
    }
    );
} catch (Exception e) {
    emitter.completeWithError(e);
}
});

return emitter;
}

```

客户端接收:

```

const eventSource = new EventSource('/api/chat_stream');

eventSource.addEventListener('content', (event) => {
    console.log('收到内容:', event.data);
    document.getElementById('answer').innerHTML += event.data;
});

eventSource.addEventListener('done', () => {
    console.log('生成完成');
    eventSource.close();
});

eventSource.addEventListener('error', (event) => {
    console.error('发生错误:', event.data);
    eventSource.close();
});

```

实现方式 2: RagService 流式回调

```

// RagService.java
public void queryStream(String question, StreamCallback callback) {
    // 1. 检索文档
    List<SearchResult> searchResults =
vectorSearchService.searchSimilarDocuments(question, topK);
    callback.onSearchResults(searchResults); // 推送检索结果

    // 2. 流式调用大模型
    Flowable<GenerationResult> result = generation.streamCall(param);

    result.blockingForEach(message -> {

```

```

        String content =
message.getOutput().getChoices().get(0).getMessage().getContent();

        if (content != null) {
            callback.onContentChunk(content); // 逐块推送内容
        }
    });

    callback.onComplete(fullContent, fullReasoning); // 推送完成信号
}

```

Controller 层封装:

```

@PostMapping(value = "/rag_stream", produces =
MediaType.TEXT_EVENT_STREAM_VALUE)
public SseEmitter ragStream(@RequestBody ChatRequest request) {
    SseEmitter emitter = new SseEmitter(180000L);

    executor.submit(() -> {
        ragService.queryStream(request.getQuestion(), new
RagService.StreamCallback() {
            @Override
            public void onSearchResults(List<SearchResult> results) {
                emitter.send(SseEmitter.event().name("search").data(results));
            }

            @Override
            public void onContentChunk(String chunk) {
                emitter.send(SseEmitter.event().name("content").data(chunk));
            }

            @Override
            public void onComplete(String fullContent, String fullReasoning) {
                emitter.send(SseEmitter.event().name("done").data("[DONE]"));
                emitter.complete();
            }

            @Override
            public void onError(Exception e) {
                emitter.completeWithError(e);
            }
        });
    });

    return emitter;
}

```

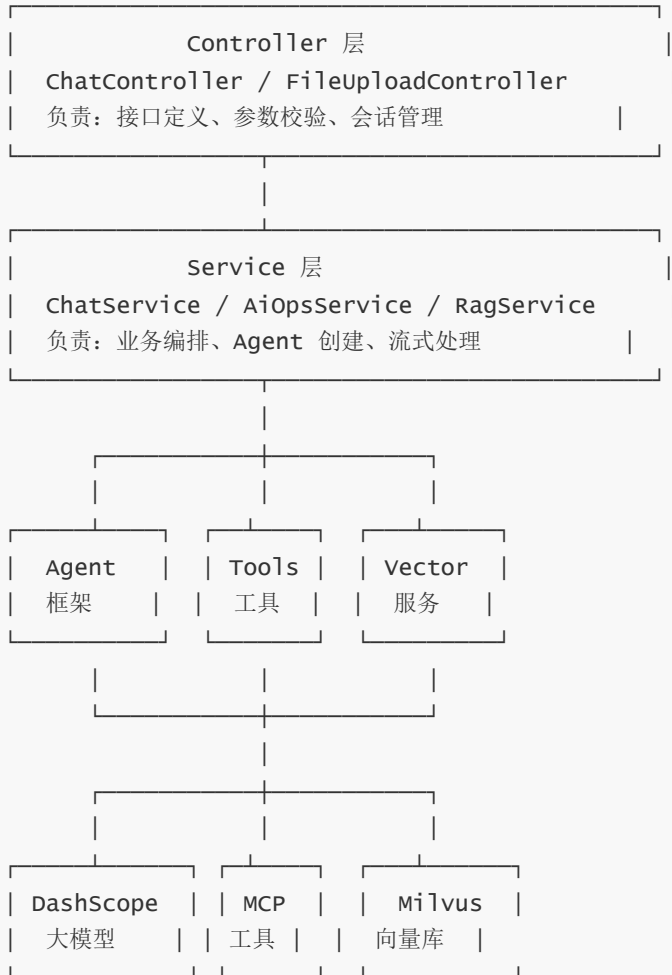
关键点总结

1. **异步执行**: 使用 `ExecutorService` 在后台线程处理, 避免阻塞 Web 线程
2. **超时设置**: `SseEmitter(180000L)` 设置 3 分钟超时
3. **事件命名**: `event().name("content")` 让前端能区分不同类型的消息
4. **异常处理**: `completeWithError(e)` 确保异常时也能正确关闭连接

Q12: 项目的整体架构是怎样的？请画出调用链路

答案：

分层架构



核心调用链路

链路 1：普通对话（支持工具调用）

```

    graph TD
      A[用户请求] --> B[POST /api/chat_stream]
      B --> C[ChatController]
      C --> D["└ 获取/创建会话 (ConcurrentHashMap)"]
      C --> E["└ 加锁 (ReentrantLock)"]
      C --> F["└ 构建历史消息 (滑动窗口)"]
      C --> G["└ 调用 ChatService"]
      G --> H[ChatService]
      H --> I["└ 创建 DashScopeApi"]
      H --> J["└ 创建 DashScopeChatModel"]
      H --> K["└ 构建系统提示词 (注入历史)"]
  
```



```

└─ 创建 ReactAgent
    └─ methodTools: dateTimeTools, internalDocsTools, queryMetricsTools
        └─ tools: MCP 工具（腾讯云日志）
            ↓
ReactAgent.stream(question)
└─ 【Thought】分析问题
└─ 【Action】调用工具
    │   └─ dateTimeTools.getCurrentDateTime()
    │   └─ internalDocsTools.queryInternalDocs()
    │       └─ vectorSearchService.searchSimilarDocuments()
    │           └─ Milvus 向量检索
    │   └─ queryMetricsTools.queryPrometheusAlerts()
    │       └─ MCP 工具（腾讯云日志查询）
└─ 【Observation】获取工具结果
└─ 【Thought】继续推理
└─ 【Answer】流式输出答案
    ↓
SseEmitter
└─ event: content → 内容块
└─ event: done → 完成信号
└─ event: error → 错误信息
    ↓
前端实时渲染

```

链路 2: AIOps 智能运维

```

用户请求
↓
POST /api/ai_ops
↓
ChatController
└─ 调用 AiOpsService
    ↓
AiOpsService
└─ 创建 PlannerAgent（规划者）
└─ 创建 ExecutorAgent（执行者）
└─ 创建 SupervisorAgent（调度者）
    ↓
SupervisorAgent.invoke(taskPrompt)
└─ 【循环开始】
└─ 调用 PlannerAgent
    │   └─ 输出计划 {"decision": "EXECUTE", "step": "..."}
└─ 调用 ExecutorAgent
    │   └─ 执行第一步
    │   └─ 调用工具（告警、日志、文档）
    │   └─ 返回反馈 {"status": "SUCCESS", "summary": "..."}
└─ 判断是否完成？
    │   └─ 否 → 回到 PlannerAgent（重新规划）
    │   └─ 是 → 输出最终报告
└─ 【循环结束】
    ↓
提取最终报告（Markdown 格式）
↓
SseEmitter 流式输出报告
↓

```

链路 3: RAG 问答

```
用户上传文档
↓
POST /api/upload
↓
FileUploadController
├─ 保存文件到 ./uploads
└─ 调用 VectorIndexService
    ↓
VectorIndexService.indexDocument()
├─ DocumentChunkService.chunkDocument()
│   └─ 切分文档 (max-size: 800, overlap: 100)
├─ VectorEmbeddingService.generateEmbedding()
│   └─ 调用 DashScope text-embedding-v4
└─ Milvus.insert()
    └─ 存储向量和元数据

---

用户提问
↓
POST /api/rag_stream
↓
RagService.queryStream()
├─ VectorSearchService.searchSimilarDocuments()
│   ├── 查询向量化
│   ├── Milvus 相似度检索 (top-k=3)
│   └─ 返回 SearchResult[]
├─ 构建上下文 (拼接检索到的文档)
├─ 构建 Prompt (注入上下文)
└─ DashScope.streamCall()
    └─ 流式生成答案
    ↓
SseEmitter 逐块推送
↓
前端实时展示
```

Q13: 为什么要做 Mock 模式？如何实现的？

答案：

为什么需要 Mock 模式？

实际开发中的问题：

1. 外部依赖不可用：

- Prometheus 还没部署
- 腾讯云日志服务需要付费
- 测试环境访问不了生产监控

2. 测试不稳定：

- 外部 API 偶尔超时
- 测试数据不可控

3. 演示需要:

- 面试时不可能连真实生产环境
- Demo 需要可预测的输出

Mock 模式的优势:

- ☒ **开发效率高**: 不依赖外部系统即可开发
- ☒ **测试稳定**: 返回固定的模拟数据
- ☒ **可演示性强**: 随时随地 Demo
- ☒ **成本低**: 不需要真实环境

实现方式

1 配置文件控制

```
# application.yml
prometheus:
  base-url: http://localhost:9090
  mock-enabled: true # 启用 Mock 模式

cls:
  mock-enabled: true # 启用 Mock 模式
```

2 条件注册 Bean

```
// QueryLogsTools.java
@Component
@ConditionalOnProperty(name = "cls.mock-enabled", havingValue = "true")
public class QueryLogsTools {
    // 只有 cls.mock-enabled=true 时才注册这个 Bean
}
```

为什么这样设计?

- `cls.mock-enabled=false` 时, 日志查询由 **MCP 服务** 提供 (真实环境)
- `cls.mock-enabled=true` 时, 日志查询由 **QueryLogsTools** 提供 (Mock 模式)

3 Mock 数据实现

```
// QueryMetricsTools.java
@Component
public class QueryMetricsTools {

    @Value("${prometheus.mock-enabled:false}")
    private boolean mockEnabled;

    @Tool(description = "Query Prometheus alerts")
    public String queryPrometheusAlerts() {
        if (mockEnabled) {
            // Mock 模式: 返回模拟告警数据
            return ""
        }
    }
}
```

```

        "status": "success",
        "data": {
            "alerts": [
                {
                    "labels": {
                        "alertname": "HighMemoryUsage",
                        "severity": "warning",
                        "instance": "api-server-1"
                    },
                    "state": "firing",
                    "activeAt": "2026-04-08T10:30:00Z",
                    "value": "85%"
                },
                {
                    "labels": {
                        "alertname": "HighCPUUsage",
                        "severity": "critical",
                        "instance": "web-server-2"
                    },
                    "state": "firing",
                    "activeAt": "2026-04-08T09:45:00Z",
                    "value": "92%"
                }
            ]
        }
    }
    """
} else {
    // 真实模式: 调用 Prometheus API
    return callPrometheusApi()
}
}

private String callPrometheusApi() {
    // 实际 HTTP 请求逻辑
    RestTemplate restTemplate = new RestTemplate();
    String url = prometheusBaseUrl + "/api/v1/alerts";
    return restTemplate.getForObject(url, String.class);
}
}

```

4 动态工具数组

```

// ChatService.java
public Object[] buildMethodToolsArray() {
    if (queryLogsTools != null) {
        // Mock 模式: 包含 QueryLogsTools
        return new Object[]{
            dateTimeTools,
            internalDocsTools,
            queryMetricsTools,
            queryLogsTools // Mock 日志工具
        };
    } else {
        // 真实模式: 不包含 QueryLogsTools (由 MCP 提供)
    }
}

```

```
        return new Object[]{
            dateTimeTools,
            internalDocsTools,
            queryMetricsTools
        };
    }
}
```

Mock vs Real 切换

开发/测试环境 (Mock) :

```
prometheus:
  mock-enabled: true
cls:
  mock-enabled: true
spring:
  ai:
    mcp:
      client:
        enabled: false # 关闭 MCP
```

生产环境 (Real) :

```
prometheus:
  base-url: https://prometheus.prod.example.com
  mock-enabled: false
cls:
  mock-enabled: false
spring:
  ai:
    mcp:
      client:
        enabled: true # 启用 MCP
      sse:
        connections:
          tencent-cls:
            url: https://mcp-api.tencent-cloud.com
            sse-endpoint: /sse/20c99d24fd931971
```

Q14: 会话历史为什么只保留最近 6 对消息?

答案:

滑动窗口设计

```
// ChatController.java
private static final int MAX_WINDOW_SIZE = 6; // 6 对 (12 条消息)

private void maintainWindow(List<Map<String, String>> history, int
maxWindowSize) {
    // 一对 = 用户消息 + 助手回复
    while (history.size() > maxWindowSize * 2) {
        history.remove(0); // 移除最早的消息
        history.remove(0);
    }
}
```

为什么需要限制？

问题 1: Token 限制

模型	最大 Context 长度
qwen3-max	8192 tokens
gpt-4	8192 tokens
claude-3	200k tokens

计算示例：

- 系统提示词：~200 tokens
- 每对对话：~100-200 tokens
- 当前问题：~50 tokens
- 回复：~500 tokens

总计 = 200 + (6对 × 150) + 50 + 500
= 200 + 900 + 50 + 500
= 1650 tokens (安全范围内)

如果不限制，10 对对话就可能超过 2000 tokens，20 对就可能接近上限。

问题 2: 推理效率

- Context 越长，模型推理越慢
- 越早的对话，对当前问题的相关性越低
- 保留最近 6 对已足够维持对话连贯性

问题 3: 内存占用

```
// 假设 1000 个活跃会话
// 每个会话保留 6 对消息
// 每条消息平均 200 字符

内存 = 1000 × 12 × 200 × 2 bytes
      ≈ 4.8 MB (可接受)

// 如果不限制，某些长对话可能达到 50 对
内存 = 1000 × 100 × 200 × 2 bytes
      ≈ 40 MB (内存压力)
```

如何选择窗口大小？

窗口大小建议：

场景	建议窗口	原因
简单问答	2-3 对	无需太多上下文
多轮对话	5-8 对	需要记住前几轮内容
长期任务	10+ 对	需要全局状态

本项目选择 6 对的原因：

- ☒ 能记住最近 3 个话题切换
- ☒ Token 使用安全 (约 1500 tokens)
- ☒ 推理速度快
- ☒ 内存占用低

更好的方案（扩展思路）

如果需要支持更长的对话历史：

方案 1：总结压缩

```
// 每 10 对消息，生成摘要
if (history.size() > 20) {
    String summary = summarize(history.subList(0, 10));
    history.clear();
    history.add(Map.of("role", "system", "content", "历史摘要: " + summary));
}
```

方案 2：持久化到数据库

```
// 内存只保留最近 6 对，但完整历史存 Redis/MySQL
sessionRepository.saveHistory(sessionId, fullHistory);
List<Map<String, String>> recentHistory = getRecentN(sessionId, 6);
```

方案 3：语义检索历史

```
// 向量化所有历史消息，检索与当前问题最相关的 k 条
List<String> relevantHistory = vectorSearch(currentQuestion, allHistory,
topK=5);
```

Q15: 如果让你优化这个项目，你会从哪些方面入手？

答案：

1 会话持久化 (Redis)

现状问题：

- 会话存在内存 (ConcurrentHashMap)
- 服务重启后会话丢失
- 多实例部署时会话不共享

优化方案：

```
// 使用 Spring Data Redis
@Service
public class RedisSessionService {

    @Autowired
    private RedisTemplate<String, SessionInfo> redisTemplate;

    private static final int TTL_HOURS = 24; // 会话过期时间 24 小时

    public SessionInfo getOrCreateSession(String sessionId) {
        String key = "session:" + sessionId;
        SessionInfo session = redisTemplate.opsForValue().get(key);

        if (session == null) {
            session = new SessionInfo();
            saveSession(sessionId, session);
        }

        return session;
    }

    public void saveSession(String sessionId, SessionInfo session) {
        String key = "session:" + sessionId;
        redisTemplate.opsForValue().set(key, session, TTL_HOURS,
TimeUnit.HOURS);
    }
}
```

优势：

- ☒ 持久化，重启不丢失
- ☒ 支持多实例部署
- ☒ 自动过期清理
- ☒ 可扩展到分布式锁

2 RAG 质量优化

现状问题:

- 只做了简单的 top-k 召回
- 没有 Rerank (重排序)
- 没有评估召回质量

优化方案:

方案 A: 引入 Reranker

```
// 两阶段检索
public List<SearchResult> searchWithRerank(String query, int topK) {
    // 第一阶段: 向量召回 top-30
    List<SearchResult> candidates = vectorSearch(query, topK * 10);

    // 第二阶段: Reranker 精排 top-3
    List<SearchResult> reranked = reranker.rerank(query, candidates, topK);

    return reranked;
}
```

Reranker 选择:

- bge-reranker-v2-m3 (中文效果好)
- cohere-rerank (英文效果好)

方案 B: Query Rewrite (查询重写)

```
// 用户问题可能表达不清楚, 让大模型先改写
String originalQuery = "内存炸了怎么办";
String rewrittenQuery = llm.rewrite(originalQuery);
// → "如何排查和处理 Java 应用内存溢出 (OOM) 问题"

List<SearchResult> results = vectorSearch(rewrittenQuery, topK);
```

方案 C: 混合检索 (Hybrid Search)

```
// 同时用向量检索和关键词检索, 融合结果
List<SearchResult> vectorResults = vectorSearch(query, topK);           // 语义相似
List<SearchResult> keywordResults = keywordSearch(query, topK);        // 关键词匹配

// 融合 (例如: 倒数排序融合)
List<SearchResult> hybridResults = rrf(vectorResults, keywordResults);
```

方案 D: 添加引用溯源

```
// 在回答中标注信息来源
```

```
String answer = ""
```

根据文档《运维手册v2.3》第5章：

> **Java** 应用内存溢出通常由以下原因引起：

- > 1. 堆内存设置过小
- > 2. 内存泄漏
- > 3. 大对象频繁创建

****来源**：**【参考资料1】运维手册v2.3 第5章第2节

```
"";
```

3 监控与可观测性

现状问题：

- 无法监控 Agent 执行过程
- 不知道哪些工具被调用了多少次
- 无法评估 RAG 召回质量

优化方案：

方案 A：日志埋点

```
@Aspect
@Component
public class ToolCallMonitor {

    @Around("@annotation(org.springframework.ai.tool.annotation.Tool)")
    public Object monitor(ProceedingJoinPoint joinPoint) throws Throwable {
        String toolName = joinPoint.getSignature().getName();
        long startTime = System.currentTimeMillis();

        try {
            Object result = joinPoint.proceed();
            long duration = System.currentTimeMillis() - startTime;

            // 记录成功调用
            logToolCall(toolName, duration, "SUCCESS");

            return result;
        } catch (Exception e) {
            // 记录失败调用
            logToolCall(toolName, System.currentTimeMillis() - startTime,
"FAILED");
            throw e;
        }
    }
}
```

方案 B：Prometheus 指标

```
@Component
public class MetricsCollector {
```

```

        private final Counter toolCallCounter = Counter.builder("tool_calls_total")
            .tag("tool", "")
            .tag("status", "")
            .register(Metrics.globalRegistry);

        private final Timer toolCallTimer = Timer.builder("tool_call_duration")
            .tag("tool", "")
            .register(Metrics.globalRegistry);

        public void recordToolCall(String toolName, boolean success, long duration)
        {
            toolCallCounter.increment();
            toolCallTimer.record(duration, TimeUnit.MILLISECONDS);
        }
    }
}

```

方案 C: RAG 召回评估

```

// 评估检索质量
public double evaluateRecall(String query, List<SearchResult> results) {
    // 计算平均相似度
    double avgScore = results.stream()
        .mapToDouble(SearchResult::getScore)
        .average()
        .orElse(0.0);

    // 记录低质量召回
    if (avgScore < 0.6) {
        logger.warn("Low recall quality for query: {}, avg_score: {}", query,
            avgScore);
    }

    return avgScore;
}

```

4 安全性优化

现状问题:

- API Key 明文写在配置文件
- 无接口限流
- 无用户认证

优化方案:

方案 A: 密钥管理

```
// 使用 Spring Cloud Config + Vault
@Configuration
public class SecurityConfig {

    @Value("${dashscope.api.key:${vault.read('secret/dashscope').apiKey}}")
    private String apiKey;

}
```

方案 B：接口限流

```
@Component
public class RateLimiter {

    private final LoadingCache<String, AtomicInteger> requestCounts =
CacheBuilder.newBuilder()
    .expireAfterWrite(1, TimeUnit.MINUTES)
    .build(new CacheLoader<String, AtomicInteger>() {
        @Override
        public AtomicInteger load(String key) {
            return new AtomicInteger(0);
        }
    });

    public boolean allowRequest(String sessionId) {
        AtomicInteger count = requestCounts.get(sessionId);
        return count.incrementAndGet() <= 10; // 每分钟最多 10 次请求
    }

}
```

方案 C：输入校验

```
@PostMapping("/chat")
public ResponseEntity<?> chat(@RequestBody @Valid ChatRequest request) {
    // 使用 JSR-303 校验
}

public class ChatRequest {
    @NotBlank(message = "问题不能为空")
    @Size(max = 1000, message = "问题长度不能超过 1000 字符")
    private String question;
}
```

5 成本优化

现状问题：

- 每次调用都用 qwen3-max（成本高）
- 没有缓存机制
- 向量检索未优化

优化方案：

方案 A：智能路由（根据问题复杂度选模型）

```

public String selectModel(String question) {
    if (isSimpleQuestion(question)) {
        return "qwen-turbo"; // 便宜、快速
    } else if (isComplexQuestion(question)) {
        return "qwen3-max"; // 贵、效果好
    } else {
        return "qwen-plus"; // 中等
    }
}

```

方案 B: 语义缓存

```

@Cacheable(value = "rag-answers", key = "#query")
public String getCachedAnswer(String query) {
    // 如果问题相似度 > 0.95, 直接返回缓存答案
    return ragService.query(query);
}

```

方案 C: 向量索引优化

```

// Milvus 索引优化
CreateIndexParam indexParam = CreateIndexParam.builder()
    .collectionName("documents")
    .fieldName("vector")
    .indexType(IndexType.HNSW) // 高性能图索引
    .metricType(MetricType.COSINE)
    .extraParam("{\"M\": 16, \"efConstruction\": 200}") // 调参
    .build();

```

第四部分：行为问题（软技能）

Q16: 开发过程中遇到过最困难的问题是什么？如何解决的？

答案：

问题描述：

在实现 AIOps 多 Agent 协作时，遇到了 **Planner Agent 无限循环** 的问题。

问题表现：

```

Planner: {"decision": "EXECUTE", "step": "查询告警"}
Executor: {"status": "SUCCESS", "summary": "..."}

Planner: {"decision": "EXECUTE", "step": "查询告警"} // 又是同一步
Executor: {"status": "SUCCESS", "summary": "..."}

Planner: {"decision": "EXECUTE", "step": "查询告警"} // 又又是同一步
...（无限循环，直到超时）

```

原因分析：

1. Planner 没有记住已执行的步骤

2. **系统提示词不够明确**，没有告诉 Planner 如何判断"何时该结束"
3. **Executor 的反馈格式不统一**，导致 Planner 无法解析

解决过程：

尝试 1：增加步骤计数器

```
int executedSteps = 0;
while (executedSteps < 10) {
    // 执行 Planner → Executor
    executedSteps++;
}
```

❌ **失败**：治标不治本，还是会重复执行

尝试 2：优化 Planner Prompt

```
String plannerPrompt = """
你已经执行过的步骤: {executed_steps}
请根据已有信息，决定下一步：
- 如果信息充足，输出 decision=FINISH
- 如果需要更多信息，输出 decision=EXECUTE 并说明原因
""";
```

✅ **有效**：Planner 能够意识到"不要重复"

尝试 3：标准化 Executor 输出格式

```
{
  "status": "SUCCESS" | "FAILED",
  "step_id": "query_alerts", // 添加步骤 ID
  "summary": "...",
  "evidence": "...",
  "nextHint": "..."
}
```

✅ **有效**：Planner 能够识别哪些步骤已完成

最终方案：

```
// 在 SupervisorAgent 的系统提示词中明确终止条件
String supervisorPrompt = """
如果发现 Planner 连续 3 次输出相同的步骤，必须终止流程，
直接输出"任务无法完成"的报告，明确告知失败原因。
""";
```

收获：

- 深入理解了 **Agent 提示词工程** 的重要性
 - 学会了如何设计 **状态机和终止条件**
 - 掌握了 **Debug 多 Agent 系统的方法**（日志、状态追踪）
-

Q17: 为什么对 AI 感兴趣？未来的学习计划是什么？

答案：

为什么对 AI 感兴趣：

1. **技术前沿性**：AI 正在改变各行各业，是未来 10 年的核心技术
2. **问题解决能力**：AI 能解决传统编程难以处理的问题（NLP、图像识别、推荐系统）
3. **实际应用价值**：从 ChatGPT 到 Copilot，AI 已经融入日常工作和生活

这个项目给我的启发：

- **AI 不是万能的**：需要工程化手段（RAG、Agent、Prompt Engineering）才能落地
- **工程能力同样重要**：再好的模型，没有好的系统设计也无法发挥价值
- **AI + 传统技术** 是趋势：向量数据库、流式处理、分布式系统等

未来学习计划：

短期（3-6 个月）：

1. **深入 RAG 技术**：
 - 学习 Reranker、Hybrid Search
 - 研究 LlamaIndex、LangChain 框架
2. **掌握 Agent 框架**：
 - 阅读 AutoGPT、BabyAGI 源码
 - 实现一个 Multi-Agent 项目
3. **提升工程能力**：
 - 学习 Kubernetes、微服务架构
 - 优化项目的可扩展性和性能

中期（6-12 个月）：

1. **模型微调**：
 - 学习 LoRA、QLoRA 微调方法
 - 尝试微调一个垂直领域模型
2. **多模态 AI**：
 - 研究图文混合的 RAG
 - 探索 GPT-4V、CLIP 等多模态模型
3. **AI 安全与评估**：
 - 学习 Prompt Injection 防御
 - 研究 AI 输出评估方法（BLEU、ROUGE、BERTScore）

长期（1-2 年）：

- 深入 Transformer 原理，阅读相关论文
- 参与开源 AI 项目（Hugging Face、LangChain）
- 尝试发表一篇 AI 应用方向的论文

第五部分：快问快答（面试常见）

Q18: 向量数据库和传统数据库有什么区别？

答案：

特性	传统数据库 (MySQL)	向量数据库 (Milvus)
数据类型	结构化数据 (表、行、列)	非结构化数据 (向量)
查询方式	精确匹配 (WHERE id=1)	相似度搜索 (top-k)
索引结构	B+ 树、哈希	HNSW、IVF、FAISS
适用场景	用户信息、订单数据	语义搜索、推荐系统
查询性能	$O(\log n)$	$O(k \log n)$

核心区别：

- 传统数据库: "这条数据是什么？" (精确匹配)
- 向量数据库: "哪些数据和这个相似？" (语义匹配)

Q19: 什么是 Prompt Engineering？

答案：

Prompt Engineering (提示词工程) 是设计和优化 AI 模型输入的技术，目的是让模型生成更准确、更有用的输出。

核心技巧：

1. Clear Instructions (明确指令)

- ✗

"告诉我关于内存的事"
- ✓

"请详细解释 Java 虚拟机的内存模型，包括堆、栈、方法区的作用"

2. Few-Shot Learning (少样本学习)

提取以下文本的关键信息：

- 示例 1:

输入: "服务器 CPU 占用 90%"

输出: {"resource": "CPU", "value": "90%", "status": "high"}
- 示例 2:

输入: "内存不足, OOM 错误"

输出: {"resource": "Memory", "value": "OOM", "status": "critical"}

现在请处理：
输入: "磁盘使用率 95%，接近满载"

3. Role Prompting (角色扮演)

你是一个资深的 **SRE** 工程师，擅长故障排查和系统优化。
请根据以下告警信息，给出专业的分析和建议。

4. Chain of Thought (思维链)

- 请一步步思考：
- 1. 首先分析告警的类型和严重程度
 - 2. 然后查询相关日志证据
 - 3. 接着搜索内部文档找到处理方案
 - 4. 最后生成分析报告

在本项目中的应用：

```
// ChatService.java - buildSystemPrompt()
String systemPrompt = """
你是一个专业的智能助手，可以：
- 当用户询问时间相关问题时，使用 getCurrentDateTime 工具
- 当用户需要查询内部文档时，使用 queryInternalDocs 工具
- 当用户需要查询告警时，使用 queryPrometheusAlerts 工具

请基于以上对话历史，回答用户的新问题。
""";
```

Q20: 如何评估一个 RAG 系统的好坏？

答案：

1 检索质量指标

指标	定义	计算方式
Recall@k	召回率	检索到的相关文档数 / 所有相关文档数
Precision@k	精确率	检索到的相关文档数 / 检索的总文档数
MRR (Mean Reciprocal Rank)	平均倒数排名	第一个相关文档的排名倒数的平均值

示例：

用户问题： "如何处理内存溢出？ "

相关文档： doc1, doc2, doc3

检索结果： doc2, doc5, doc1, doc7

- Recall@3 = 2/3 = 67% (检索到 doc2, doc1)
- Precision@3 = 2/3 = 67%
- MRR = 1/1 = 1.0 (doc2 排第 1)

2 生成质量指标

指标	说明
Faithfulness	答案是否忠实于检索到的文档（不编造）
Answer Relevance	答案是否回答了用户问题
Context Relevance	检索到的文档是否与问题相关

3 端到端评估

```
// 人工评估（最准确但成本高）
public void evaluate() {
    List<Question> testCases = loadTestCases();

    for (Question q : testCases) {
        String answer = ragService.query(q.getQuestion());

        // 人工打分（1-5 分）
        int score = humanEvaluate(answer, q.getGroundTruth());

        logger.info("Question: {}, Score: {}", q.getQuestion(), score);
    }
}
```

4 在线监控

```
// 记录低相似度召回
if (avgSimilarity < 0.6) {
    logger.warn("Low similarity for query: {}", query);
    metrics.incrementCounter("low_similarity_queries");
}

// 记录用户反馈
if (userFeedback == "thumbs_down") {
    logger.warn("Negative feedback for query: {}", query);
}
```

好了！这份 QA 文档涵盖了从基础到进阶的所有重要问题。祝你面试顺利！👉

📌 附录：面试前自检清单

✅ 必须能流利回答的问题

- ☐ 项目整体介绍（1 分钟版本）
- ☐ RAG 是什么？为什么需要 RAG？
- ☐ Agent 是什么？与传统 AI 的区别？
- ☐ 向量检索的原理
- ☐ 多 Agent 协作的设计

✓ 必须能画出的图

- ☐ 项目整体架构图
- ☐ RAG 工作流程图
- ☐ AIOps 多 Agent 协作流程图
- ☐ 会话管理的并发控制设计

✓ 必须能演示的功能

- ☐ 上传文档并提问 (RAG 问答)
- ☐ 普通对话 (工具调用)
- ☐ AIOps 智能运维 (生成报告)
- ☐ 流式输出效果

✓ 必须能讲清楚的优化点

- ☐ 会话持久化 (Redis)
- ☐ RAG 质量优化 (Reranker)
- ☐ 监控与可观测性
- ☐ 成本优化 (模型路由、缓存)

最后建议:

1. 把这份 QA 打印出来, 每天看一遍
2. 找同学模拟面试, 练习口头表达
3. 准备好 Demo 环境, 确保能随时演示
4. 主动提及项目亮点, 不要等面试官问

祝你拿到 offer! 🎉